

```

# =====
# NFL BETTING MODEL
# HISTORICAL TRAINING PIPELINE - STEP 1
# =====

library(nflverse)
library(nflfastR)
library(data.table)

DATA_DIR <- "data"
PROCESSED_DIR <- file.path(DATA_DIR, "processed")

dir.create(DATA_DIR, showWarnings = FALSE)
dir.create(PROCESSED_DIR, showWarnings = FALSE)

# -----
# Load one season
# -----

load_season <- function(season) {

  message("Loading season ", season, "...")

  pbp <- nflfastR::load_pbp(season)

  message(
    "Loaded ",
    nrow(pbp),
    " plays."
  )

  pbp
}

# -----
# Create completed-game results
# -----

build_game_results <- function(pbp) {

  games <- unique(
    pbp[
      !is.na(home_team) &
      !is.na(away_team),
      .(
        game_id,
        season,
        week,
        game_date,
        home_team,
        away_team,
        home_score,
        away_score
      )
    ]
  )

  games <- as.data.table(games)

  games <- games[
    !is.na(home_score) &
    !is.na(away_score)
  ]
}

```

```

games[
  'margin := home_score - away_score
]

games[
  'home_win := as.integer(margin > 0)
]

games[
  'tie := as.integer(margin == 0)
]

games[
  'total_points := home_score + away_score
]

setorder(
  games,
  game_date,
  game_id
)

games
}

# -----
# Build team game records
# -----

build_team_games <- function(games) {
  home <- games[
    '(
      game_id,
      season,
      week,
      game_date,
      team = home_team,
      opponent = away_team,
      home = 1L,
      points_for = home_score,
      points_against = away_score,
      margin = home_score - away_score,
      win = as.integer(home_score > away_score)
    )
  ]

  away <- games[
    '(
      game_id,
      season,
      week,
      game_date,
      team = away_team,
      opponent = home_team,
      home = 0L,
      points_for = away_score,
      points_against = home_score,
      margin = away_score - home_score,

```

```

    win = as.integer(away_score > home_score)
  )
]

team_games <- rbind(
  home,
  away
)

setorder(
  team_games,
  team,
  game_date,
  game_id
)

team_games
}

# -----
# Leakage-safe rolling features
#
# shift(1) means today's prediction only uses information
# available BEFORE today's game.
# -----

add_pregame_features <- function(team_games) {
  setorder(
    team_games,
    team,
    game_date,
    game_id
  )

  team_games[
    ,
    games_played := seq_len(.N) - 1L,
    by = team
  ]

  team_games[
    ,
    prior_wins := shift(
      cumsum(win),
      1L,
      fill = 0
    ),
    by = team
  ]

  team_games[
    ,
    prior_points_for := shift(
      cumsum(points_for),
      1L,
      fill = 0
    ),
    by = team
  ]

  team_games[
    ,
    prior_points_against := shift(
      cumsum(points_against),

```

```

      1L,
      fill = 0
    ),
    by = team
  ]

team_games[
  ,
  prior_margin_total := shift(
    cumsum(margin),
    1L,
    fill = 0
  ),
  by = team
]

team_games[
  ,
  win_rate := fifelse(
    games_played > 0,
    prior_wins / games_played,
    0.5
  )
]

team_games[
  ,
  avg_points_for := fifelse(
    games_played > 0,
    prior_points_for / games_played,
    22
  )
]

team_games[
  ,
  avg_points_against := fifelse(
    games_played > 0,
    prior_points_against / games_played,
    22
  )
]

team_games[
  ,
  avg_margin := fifelse(
    games_played > 0,
    prior_margin_total / games_played,
    0
  )
]

team_games
}

# -----
# Create one row per game
#
# Home and away pregame information are joined together.
# -----

build_training_rows <- function(team_games) {

  home <- team_games[
    home == 1L,

```

```

.(
  game_id,
  season,
  week,
  game_date,
  home_team = team,

  home_games_played = games_played,
  home_win_rate = win_rate,
  home_avg_points_for = avg_points_for,
  home_avg_points_against = avg_points_against,
  home_avg_margin = avg_margin
)
]

away <- team_games[
  home == 0L,
  .(
    game_id,
    away_team = team,

    away_games_played = games_played,
    away_win_rate = win_rate,
    away_avg_points_for = avg_points_for,
    away_avg_points_against = avg_points_against,
    away_avg_margin = avg_margin
  )
]

training <- merge(
  home,
  away,
  by = "game_id",
  all = FALSE
)

# Target variables
results <- unique(
  team_games[
    ,
    .(
      game_id,
      home_score = fifelse(
        home == 1L,
        points_for,
        NA_real_
      ),
      away_score = fifelse(
        home == 0L,
        points_for,
        NA_real_
      )
    )
  ],
  by = "game_id"
)

home_scores <- team_games[
  home == 1L,
  .(
    game_id,
    home_score = points_for
  )
]

```

```

away_scores <- team_games[
  home == 0L,
  .(
    game_id,
    away_score = points_for
  )
]

training <- merge(
  training,
  home_scores,
  by = "game_id",
  all.x = TRUE
)

training <- merge(
  training,
  away_scores,
  by = "game_id",
  all.x = TRUE
)

training[
  ,
  'actual_margin := home_score - away_score
]

training[
  ,
  'home_win := as.integer(
    actual_margin > 0
  )
]

training[
  ,
  'actual_total := home_score + away_score
]

setorder(
  training,
  game_date,
  game_id
)

training
}

# -----
# Process one season
# -----

process_season <- function(season) {
  pbp <- load_season(season)
  games <- build_game_results(pbp)
  team_games <- build_team_games(games)
  team_games <- add_pregame_features(team_games)
  training <- build_training_rows(team_games)
  output_file <- file.path(

```

```

    PROCESSED_DIR,
    paste0(
      "training_",
      season,
      ".rds"
    )
  )

  saveRDS(
    training,
    output_file
  )

  message(
    "Saved ",
    nrow(training),
    " games to ",
    output_file
  )

  rm(
    pbp,
    games,
    team_games,
    training
  )

  gc()

  invisible(
    output_file
  )
}
test_2025 <- process_season(2025)

message("Historical pipeline loaded successfully.")
source("historical_pipeline.R", echo = FALSE)

```